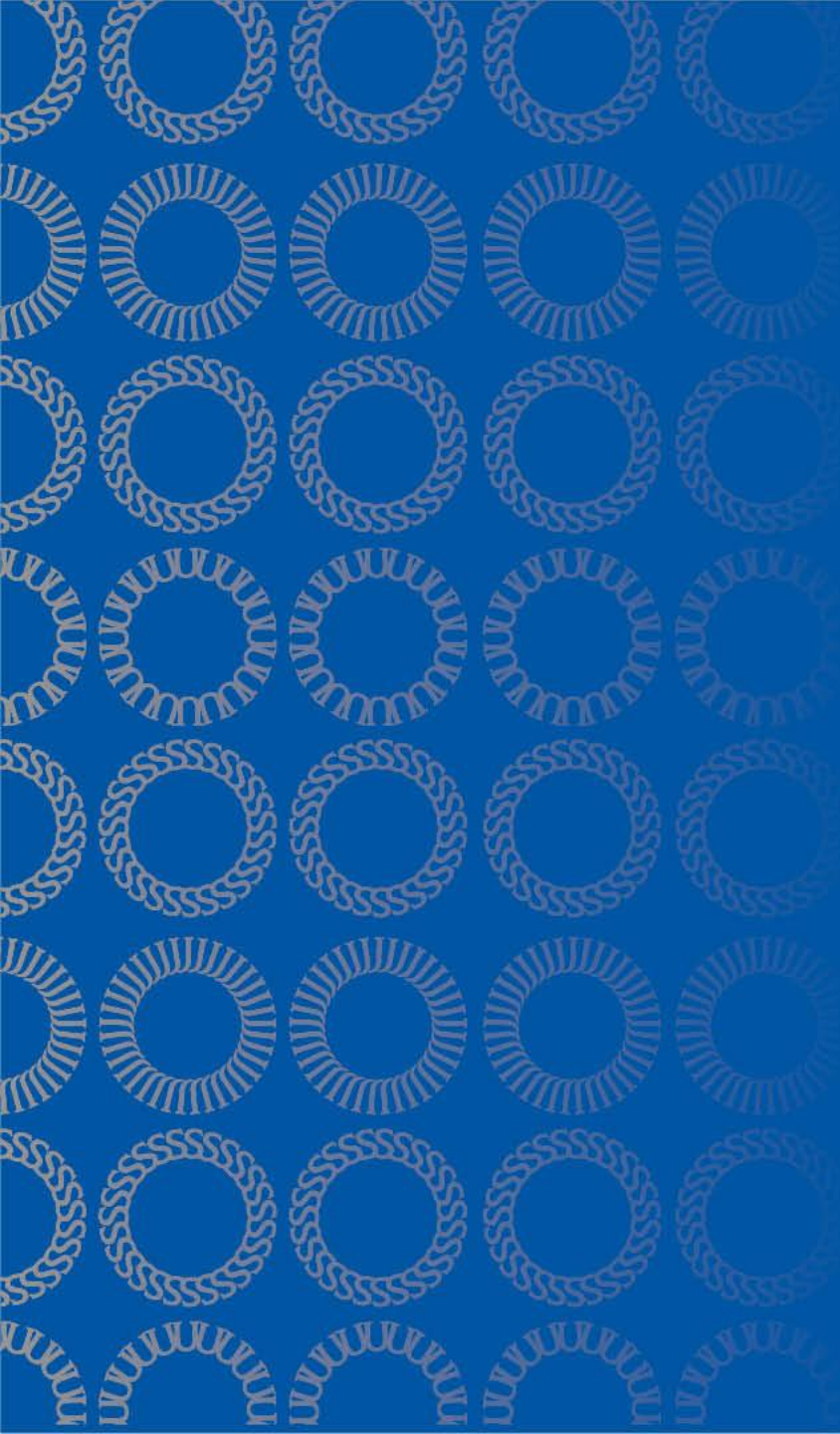




DATA 255

Deep Learning and Computer Vision in Applied Data Intelligence

Sept 2, 2026

Taehee Jeong, Ph.D.

San Jose State University

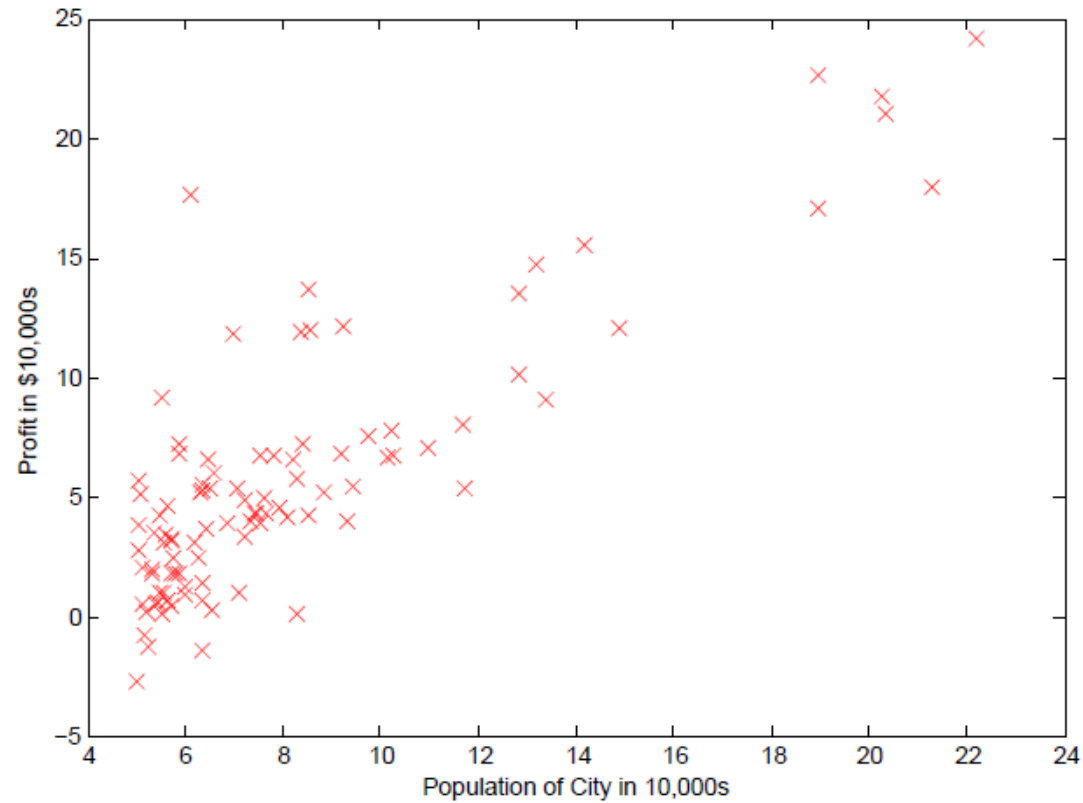
Recap Last lecture

- Linear Regression
- Logistic Regression
- Deep neural network
- Weight Initialization

Linear regression

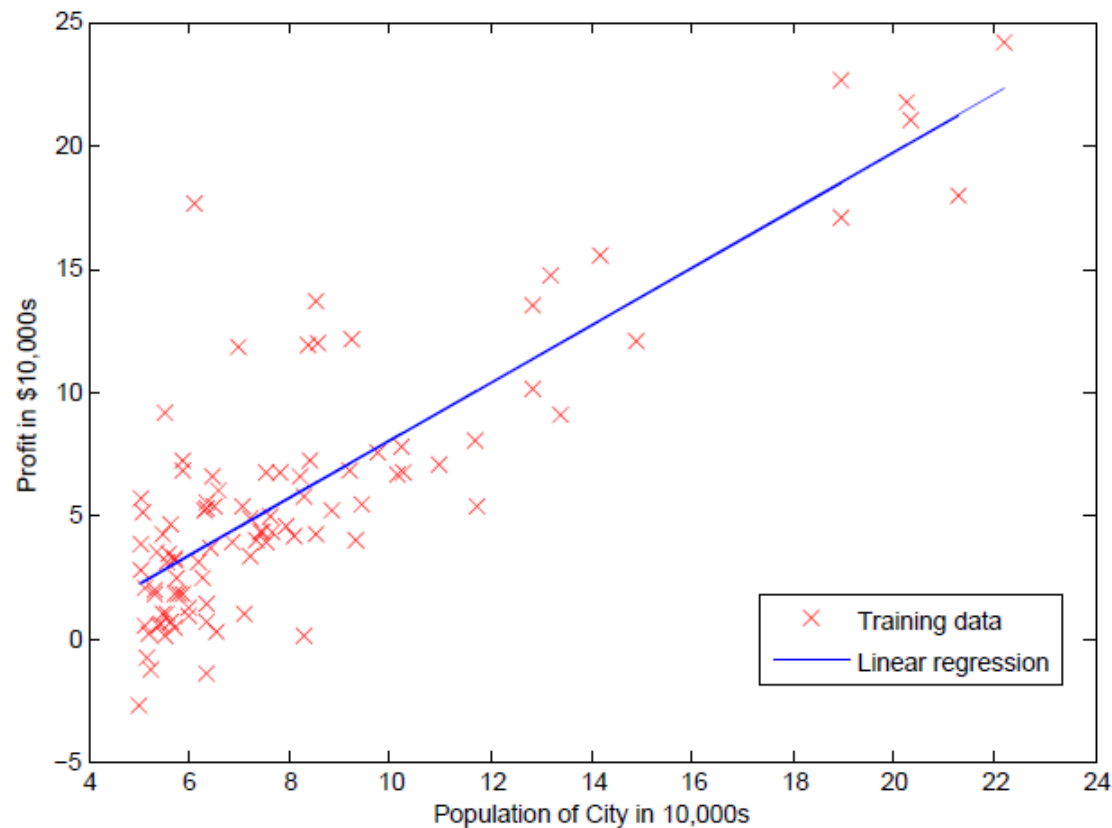
- Supervised learning
- Regression: predict continuous numerical values

Linear regression



Source: *Machine Learning*, Andrew Ng

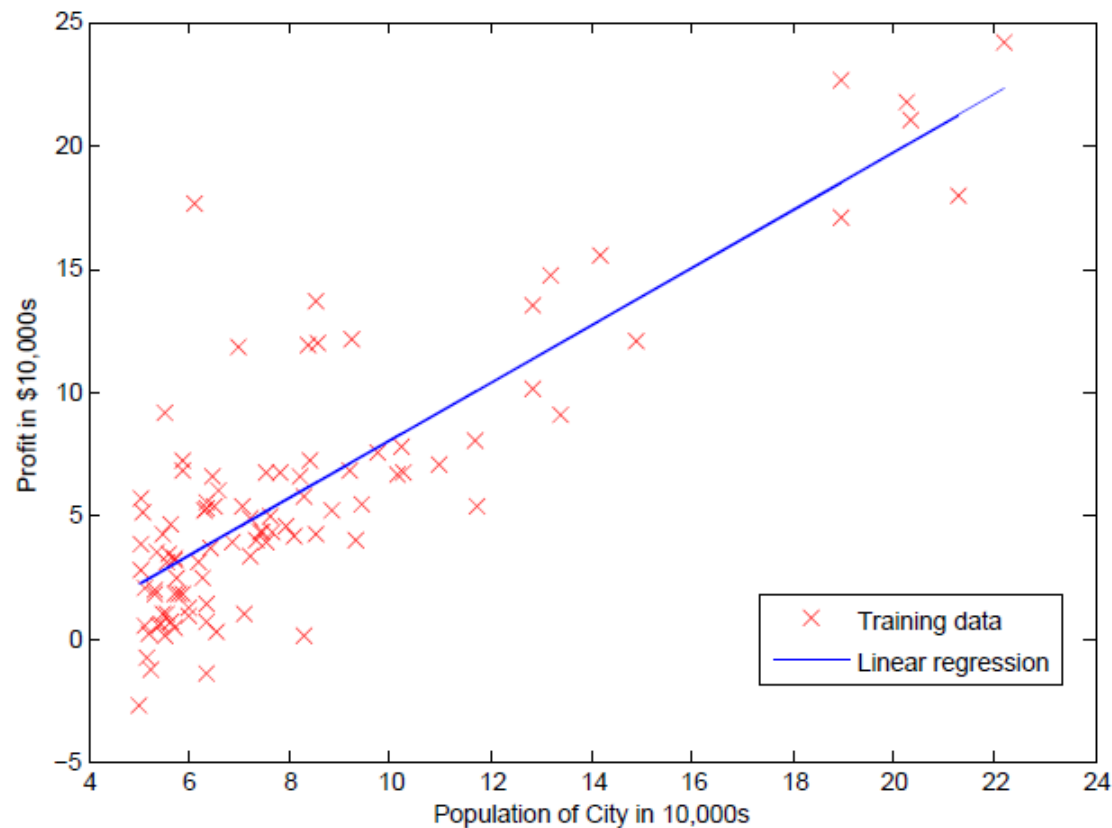
Linear regression



$$y = a \cdot x + b$$

- *a: slope*
- *b: bias*

Linear regression

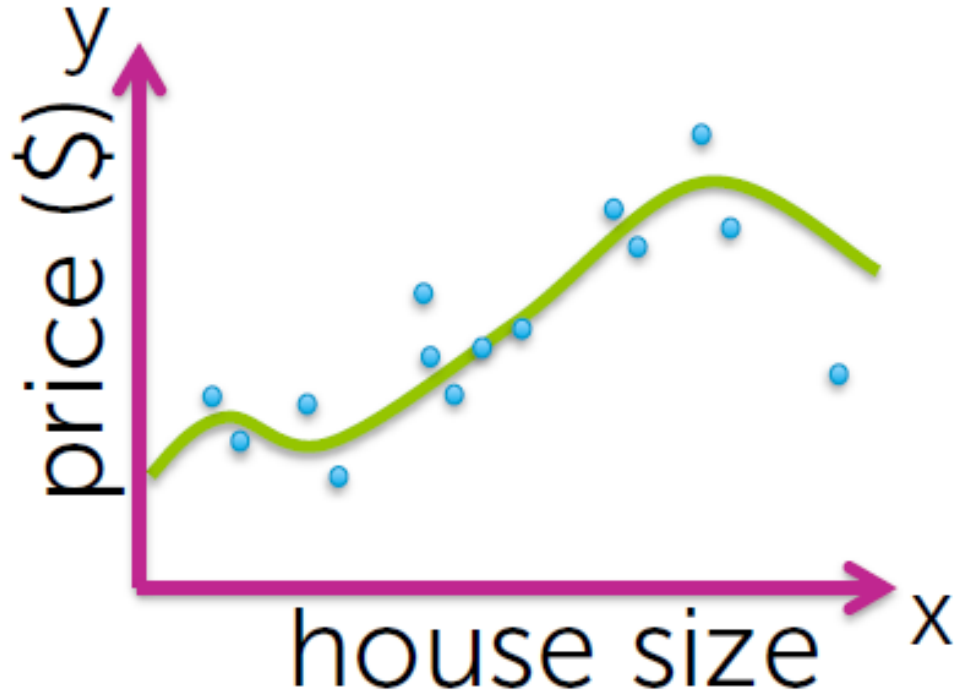


$$y = a \cdot x + b$$

$$\hat{y} = W_0 + W_1 \cdot x$$

- W_0, W_1 : Weights

Multiple variables Linear regression



More complex relationships than just a line.

$$\hat{y} = W_0 + W_1 \cdot x_1 + W_2 \cdot x_2 + \dots + W_n \cdot x_n$$

Binary classification

- The output of classification is a categorical value instead of a continuous numerical value.
- The output of binary classification is 1 or 0.
- For example,
 - Email: Spam / Not spam,
 - Online transaction: Fraudulent (Yes / No)

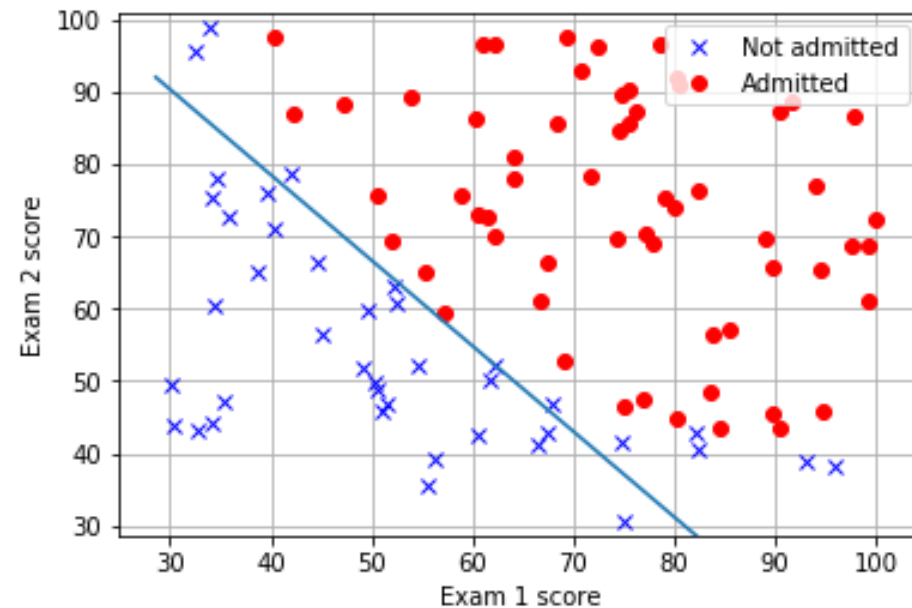
Logistic Regression

- Logistic regression is commonly used for binary classification.

Given x , the probability if $y = 1$

$$P(y = 1|x)$$

$$\hat{y} = \begin{cases} 1 & \text{if } P \geq 0.5 \\ 0 & \text{if } P < 0.5 \end{cases}$$



Logistic Regression

Given x , the probability if $y = 1$

$$\hat{y} = P(y = 1|x)$$

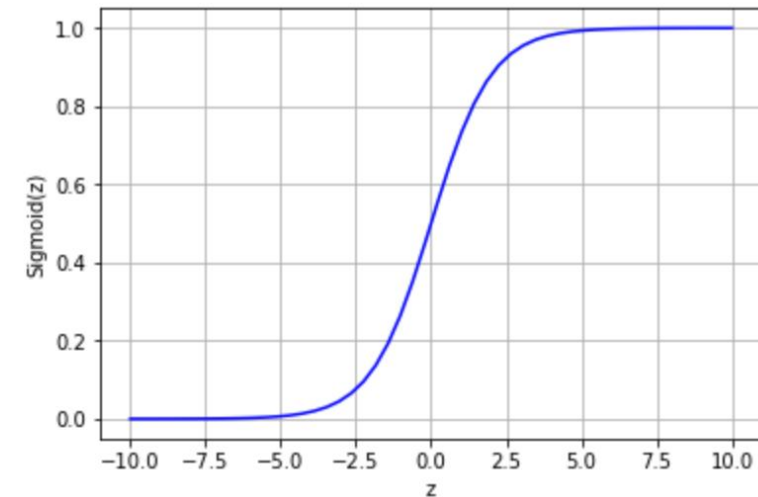
$$P = \sigma(z)$$

$$z = W_0 + W_1 \cdot x_1 + W_2 \cdot x_2 + \dots$$

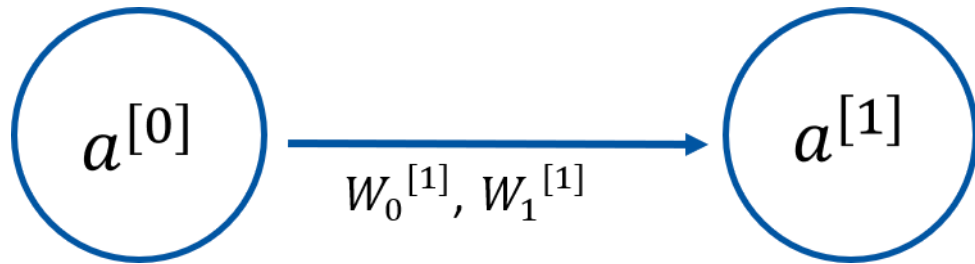
$$P = \frac{1}{1 + \exp(-(W_0 + W_1 \cdot x_1 + W_2 \cdot x_2))}$$

Sigmoid function

$$\sigma(z) = \frac{1}{1 + \exp(-z)}$$



Univariate logistic regression: Two neurons



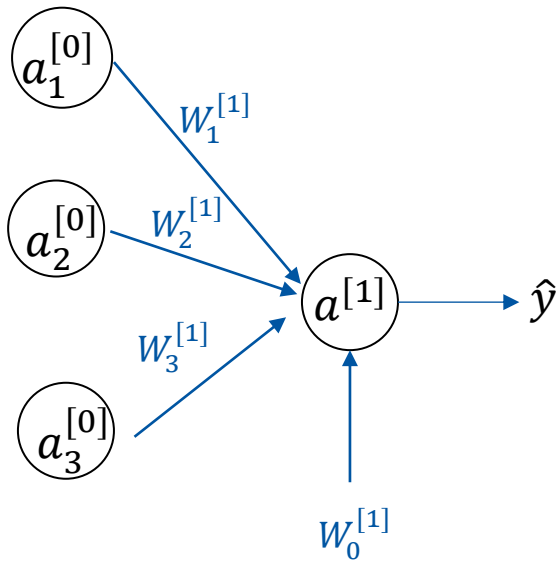
Input neuron: $x \rightarrow a^{[0]}$

$$Z^{[1]} = W_0^{[1]} + W_1^{[1]} \cdot a^{[0]}$$

Output neuron: $a^{[1]} = \sigma(Z^{[1]})$

$$\sigma(z) = \frac{1}{1 + \exp(-z)}$$

Multivariate Logistic Regression

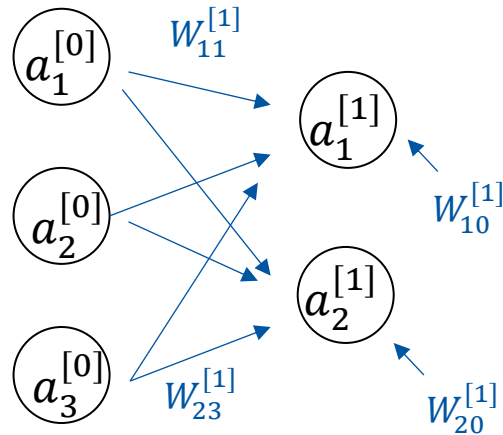


$$Z^{[1]} = W_0^{[1]} \cdot 1 + W_1^{[1]} \cdot a_1^{[0]} + W_2^{[1]} \cdot a_2^{[0]} + W_3^{[1]} \cdot a_3^{[0]}$$

$$a^{[1]} = \sigma(Z^{[1]})$$

$$\hat{y} = \begin{cases} 1 & \text{if } a^{[1]} \geq 0.5 \\ 0 & \text{if } a^{[1]} < 0.5 \end{cases}$$

Multiclass classifier



Input layer

Output layer

$$Z_1^{[1]} = W_{10}^{[1]} + W_{11}^{[1]} \cdot a_1^{[0]} + W_{12}^{[1]} \cdot a_2^{[0]} + W_{13}^{[1]} \cdot a_3^{[0]}$$

$$Z_2^{[1]} = W_{20}^{[1]} + W_{21}^{[1]} \cdot a_1^{[0]} + W_{22}^{[1]} \cdot a_2^{[0]} + W_{23}^{[1]} \cdot a_3^{[0]}$$

$$a_1^{[1]} = \sigma(Z_1^{[1]})$$

$$a_2^{[1]} = \sigma(Z_2^{[1]})$$

Hidden Layer

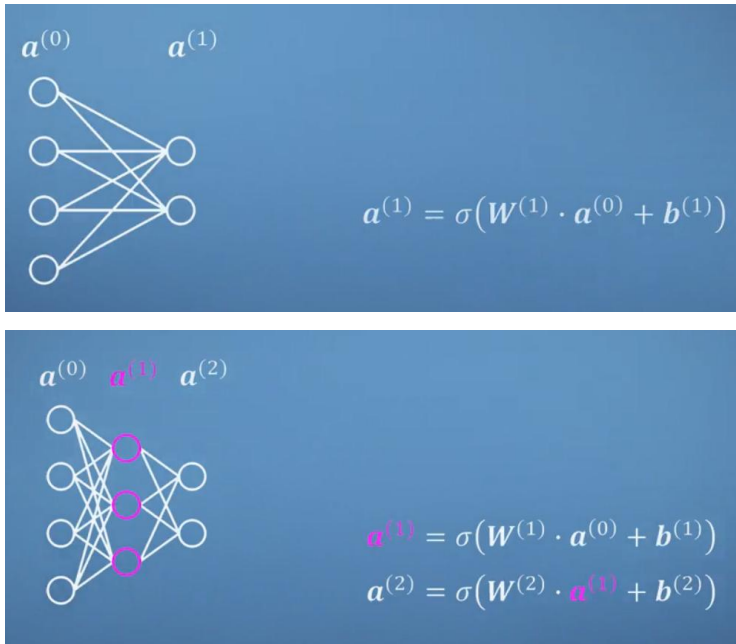


image: Mathematics for Machine Learning

- Another layer (hidden layer) can be inserted between input layer and output layer.
- It works well if there is no direct (or strong) correlation between input layer and output layer.

Neural network

- Machine Learning : No Hidden layer
- (Shallow) Neural Network : One hidden layer
- Deep Neural network: More than two hidden layers

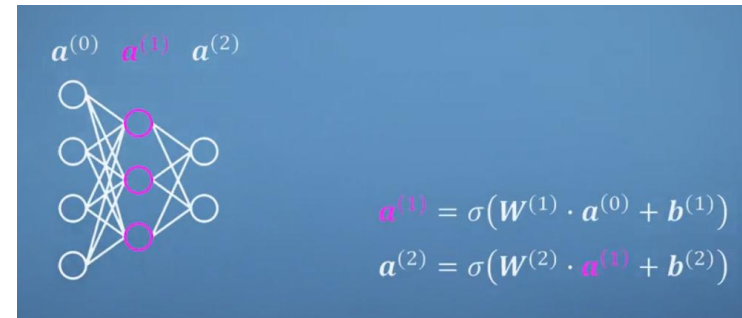
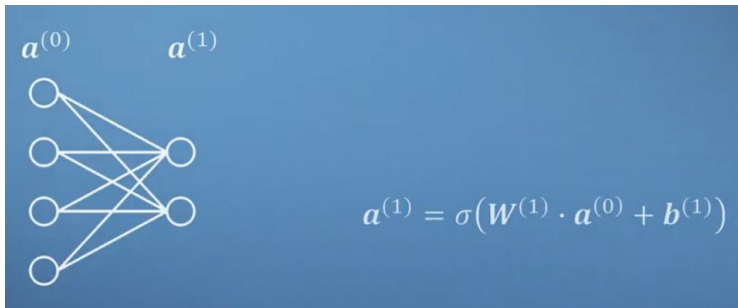
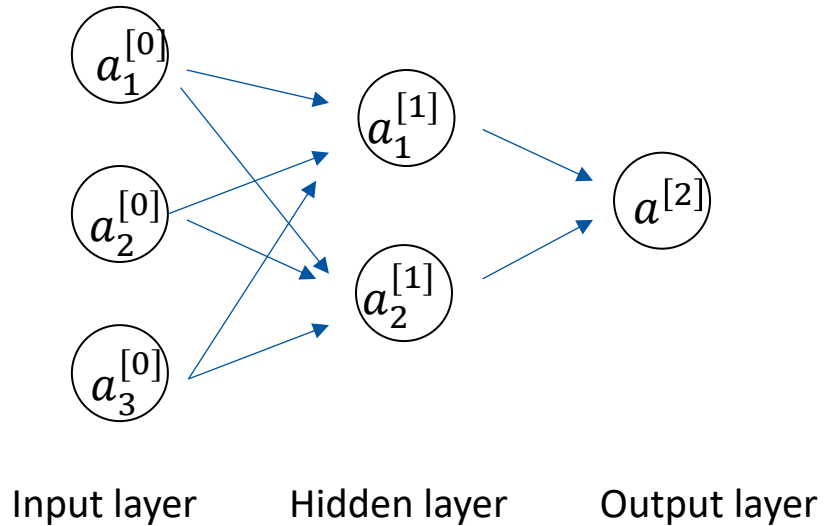


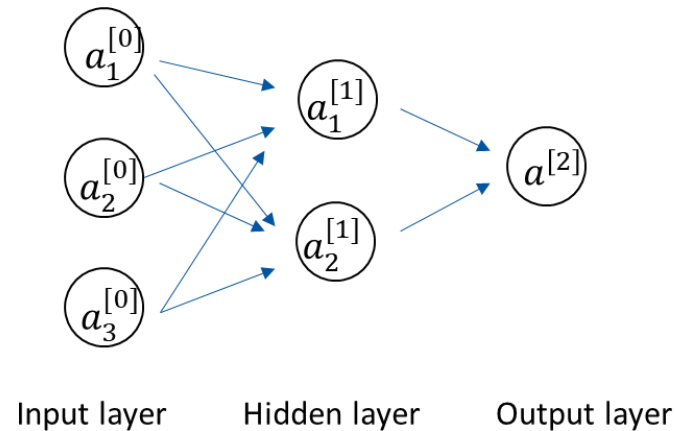
image: Mathematics for Machine Learning

Shallow Neural Network



- Input layer : 3 neurons
- Hidden layer : 2 neurons
- output layer : 1 neurons

Shallow Neural Network



- Input layer : 3 neurons
- Hidden layer : 2 neurons
- output layer : 1 neurons

Hidden layer

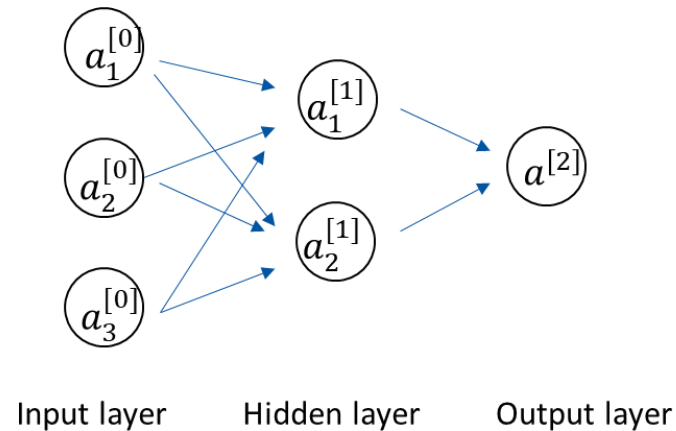
$$Z_1^{[1]} = W_{10}^{[1]} + W_{11}^{[1]} \cdot a_1^{[0]} + W_{12}^{[1]} \cdot a_2^{[0]} + W_{13}^{[1]} \cdot a_3^{[0]}$$

$$Z_2^{[1]} = W_{20}^{[1]} + W_{21}^{[1]} \cdot a_1^{[0]} + W_{22}^{[1]} \cdot a_2^{[0]} + W_{23}^{[1]} \cdot a_3^{[0]}$$

$$a_1^{[1]} = \sigma(Z_1^{[1]})$$

$$a_2^{[1]} = \sigma(Z_2^{[1]})$$

Shallow Neural Network



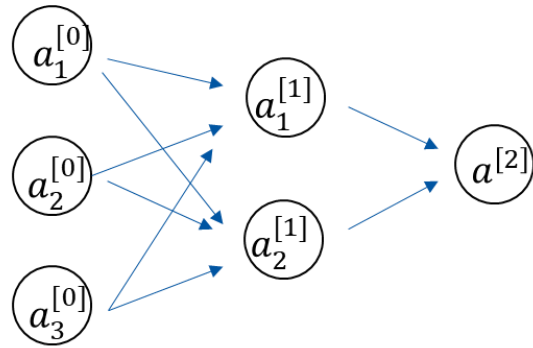
- Input layer : 3 neurons
- Hidden layer : 2 neurons
- output layer : 1 neurons

Output layer

$$Z^{[2]} = W_{10}^{[2]} + W_{11}^{[2]} \cdot a_1^{[1]} + W_{12}^{[2]} \cdot a_2^{[1]}$$

$$a^{[2]} = \sigma(Z^{[2]})$$

Neural network with matrix form



Input layer Hidden layer Output layer

- Input layer : 3 neurons
- Hidden layer : 2 neurons
- output layer : 1 neurons

Hidden layer

$$Z_1^{[1]} = W_{10}^{[1]} + W_{11}^{[1]} \cdot a_1^{[0]} + W_{12}^{[1]} \cdot a_2^{[0]} + W_{13}^{[1]} \cdot a_3^{[0]}$$

$$Z_2^{[1]} = W_{20}^{[1]} + W_{21}^{[1]} \cdot a_1^{[0]} + W_{22}^{[1]} \cdot a_2^{[0]} + W_{23}^{[1]} \cdot a_3^{[0]}$$

$$a_1^{[1]} = \sigma(Z_1^{[1]})$$

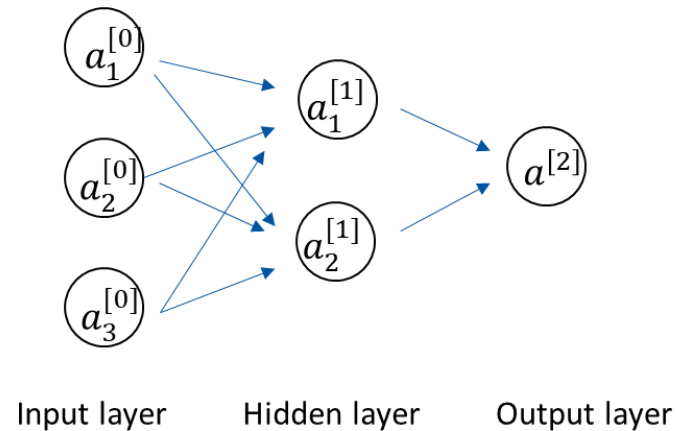
$$a_2^{[1]} = \sigma(Z_2^{[1]})$$

$$W^{[1]} : [2,4]$$

$$W^{[1]} : [n_h, n_x+1]$$

(hidden-node, input-node)

Neural network: matrix dimension



- Input layer : 3 neurons
- Hidden layer : 2 neurons
- output layer : 1 neurons

Output layer

$$Z^{[2]} = W_{10}^{[2]} + W_{11}^{[2]} \cdot a_1^{[1]} + W_{12}^{[2]} \cdot a_2^{[1]}$$

$$a^{[2]} = \sigma(Z^{[2]})$$

$$W^{[2]} : [1,3]$$

$$W^{[2]} : [n_o, n_h+1]$$

(output-node, hidden-node)

Initialize weights

Weights should be initialized randomly rather than to all zeros.

If weights are initialized to be zero, then, each neuron in the first hidden layer will perform the same computation. So even after multiple iterations of gradient descent, each neuron in the layer will be computing the same thing as other neurons.

Weights initialization

Need to break symmetry.

If $W^{[1]}$, $W_0^{[1]}$, $W^{[2]}$, $W_0^{[2]}$ are started with zero,

Then, $Z^{[1]}$, $Z^{[2]}$ become zero.

$a^{[1]}$, $a^{[2]}$ become 0.5.

$dW^{[2]}$ is same

$a^{[1]}$, $a^{[2]}$ will always get same updated!

Small random numbers

$W = 0.01 * \text{np.random.randn}(D, H)$

samples from a zero mean, unit standard deviation Gaussian

Randomness

Randomness shows up in many places: parameter initialization, dropout, data ordering, etc.

For reproducibility, we recommend you always pass in a different random seed for each use of randomness.

Determinism is particularly useful when debugging, so you can hunt down the bug.

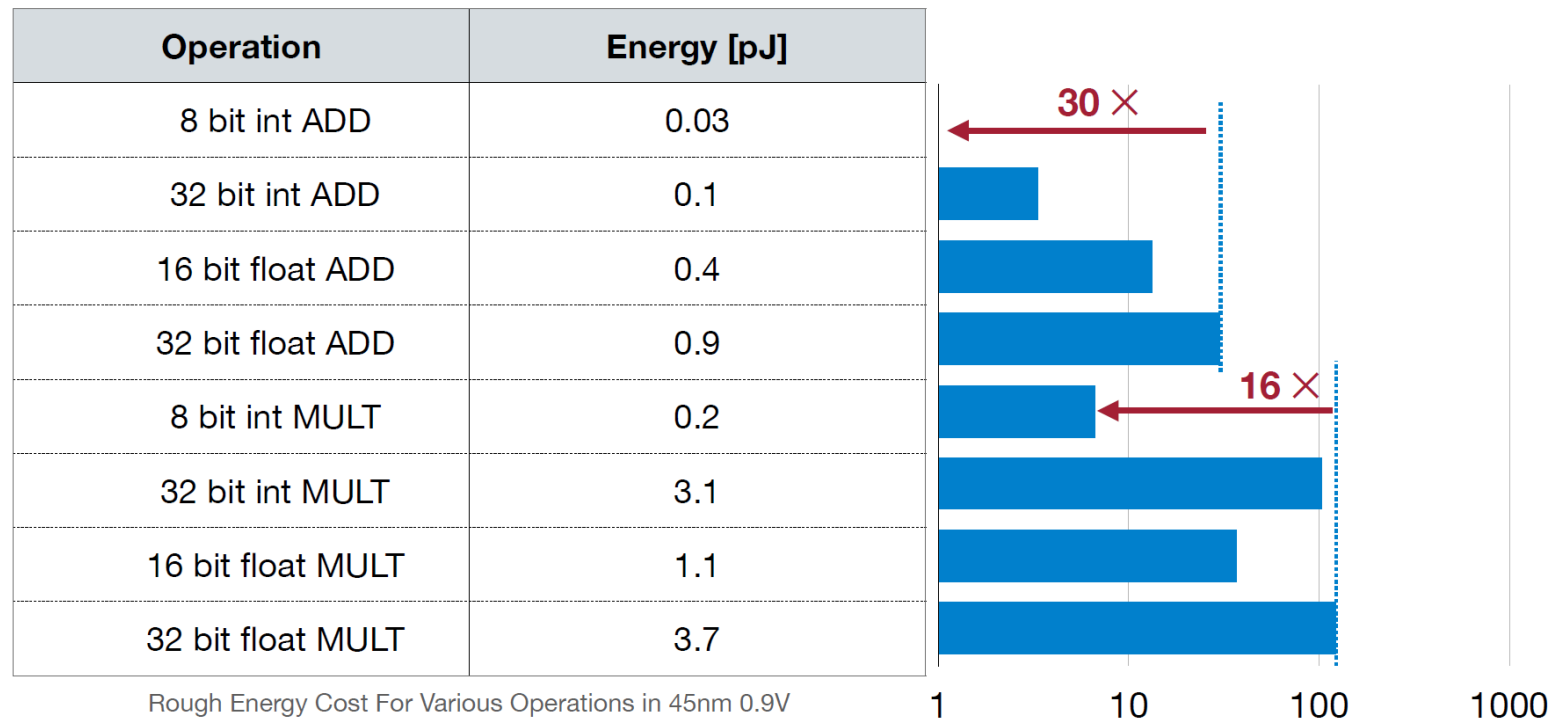
```
# set random seed  
seed = 0  
torch.manual_seed(seed)
```

Today's Topics(1)

- Quantization

Low Bit Operations are Cheaper

Less Bit-Width → Less Energy



Integer

- Unsigned Integer
 - n -bit Range: $[0, 2^n - 1]$
- Signed Integer
 - n -bit Range: $[-2^{n-1}, 2^{n-1} - 1]$

0	0	1	1	0	0	0	1
x	x	x	x	x	x	x	x

$$2^7 + 2^6 + 2^5 + 2^4 + 2^3 + 2^2 + 2^1 + 2^0 = 49$$

Sign Bit

1	0	1	1	0	0	0	1
	x	x	x	x	x	x	x

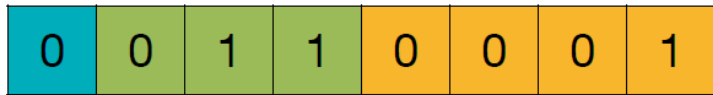
$$- 2^6 + 2^5 + 2^4 + 2^3 + 2^2 + 2^1 + 2^0 = -49$$

Fixed-Point Number



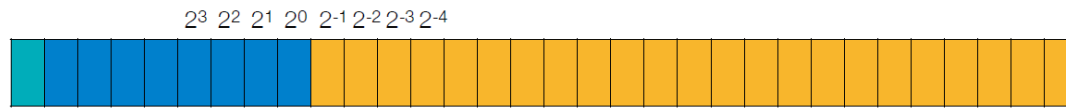
Integer . Fraction

“Decimal” Point



$$\begin{array}{cccccccc} \times & \times & \times & \times & \times & \times & \times & \times \\ 0 & 0 & 1 & 1 & 0 & 0 & 0 & 1 \\ -2^3 & +2^2 & +2^1 & +2^0 & +2^{-1} & +2^{-2} & +2^{-3} & +2^{-4} \end{array} = 3.0625$$

Floating-Point Number



Sign 1 bit **Exponent 8 bit**

23 bit Fraction (significant / mantissa)

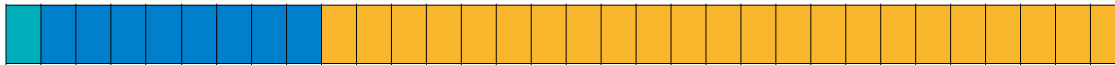
A fixed-point number keeps the decimal point at a set position, while a floating-point number allows the decimal point to move based on the value's scale.

- Sign: Indicates whether the number is positive or negative.
- Significant (Mantissa): Represents the precision bits of the number.
- Exponent: Determines the scale of the number.

Floating-Point Number

Exponent Width → Range; Fraction Width → Precision

[IEEE 754](#) Single Precision 32-bit Float (IEEE FP32)



[IEEE 754](#) Half Precision 16-bit Float (IEEE FP16)



[Google](#) Brain Float (BF16)



Exponent (bits)	Fraction (bits)	Total (bits)
8	23	32
5	10	16
8	7	16

Floating-Point Number

Exponent Width → Range; Fraction Width → Precision

[IEEE 754](#) Single Precision 32-bit Float (IEEE FP32)



Exponent
(bits)

8

Fraction
(bits)

23

Total
(bits)

32

[IEEE 754](#) Half Precision 16-bit Float (IEEE FP16)



5

10

16

[Nvidia](#) FP8 (E4M3)



* FP8 E4M3 does not have INF, and $S.1111.111_2$ is used for NaN.
* Largest FP8 E4M3 normal value is $S.1111.110_2 = 448$.

4

3

8

[Nvidia](#) FP8 (E5M2) for gradient in the backward



* FP8 E5M2 have INF ($S.11111.00_2$) and NaN ($S.11111.XX_2$).
* Largest FP8 E5M2 normal value is $S.11110.11_2 = 57344$.

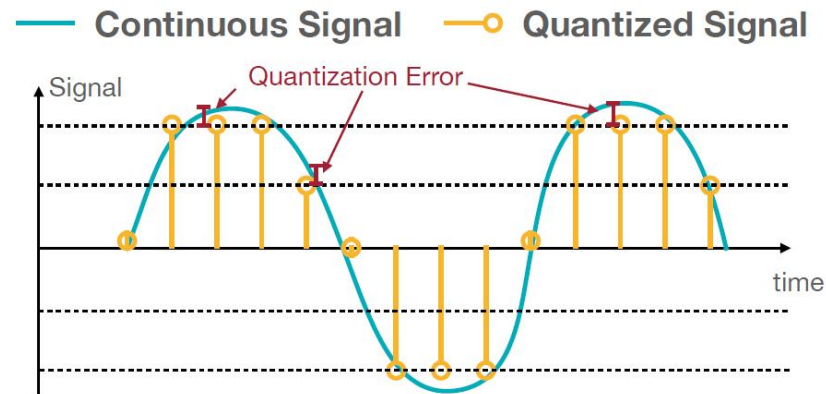
5

2

8

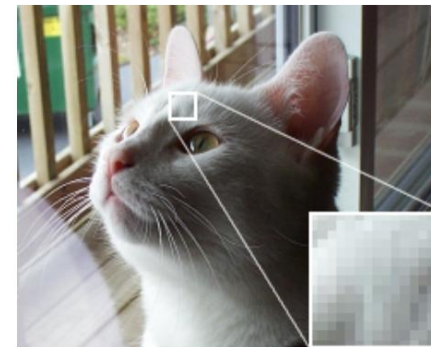
Quantization

Quantization is the process of constraining an input from a continuous or otherwise large set of values to a discrete set

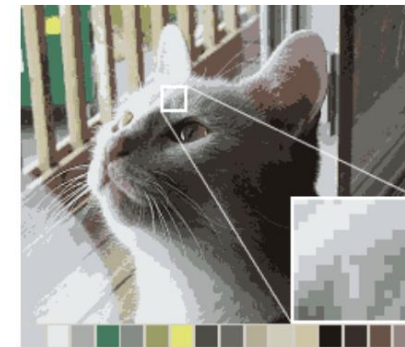


The difference between an input value and its quantized value is referred to as quantization error.

Original Image



16-Color Image

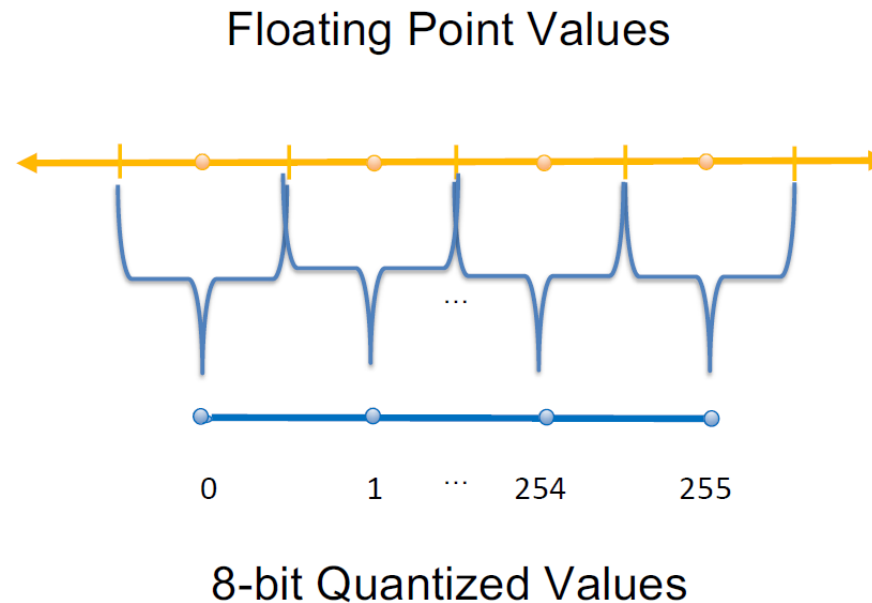


Images are in the public domain.

“Palettization”

Quantization

quantization is a linear mapping from floating point values to quantized integer values



Today's Topics(2)

- Activation function

Why we need activation functions in NN?

- Activation functions in neural networks introduce **non-linearity**, enabling the network to learn complex patterns by transforming the weighted sum of inputs into an output signal.
- They determine whether a neuron should be activated (and thus contribute to the network's output) based on the input it receives.
- Without activation functions, neural networks would only be capable of learning linear relationships

Activation functions in Neural Network

- Sigmoid
- Tanh
- ReLU
- Leaky ReLU
- ELU
- Softmax

Sigmoid

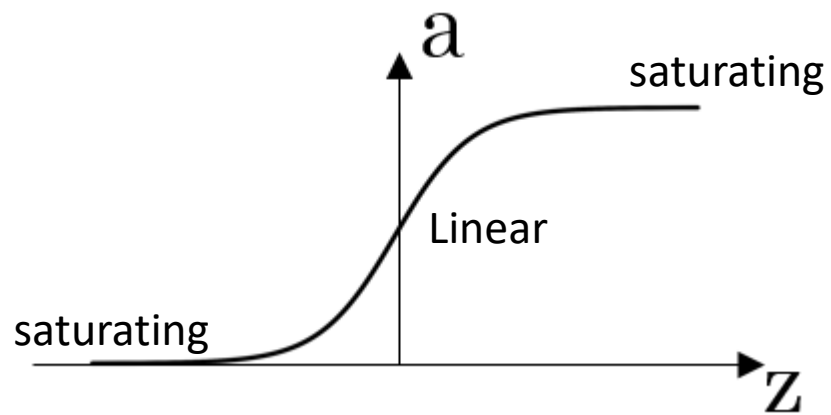


image: Deep learning, Andrew Ng

$$a = \sigma(z) = \frac{1}{1 + \exp(-z)}$$

- Sigmoid neurons can be saturated when z is very large or very small.
- The saturation leads vanishing gradients.
- The output is not zero-centered (0~1).
- $\exp()$ is computationally expensive.

tanh

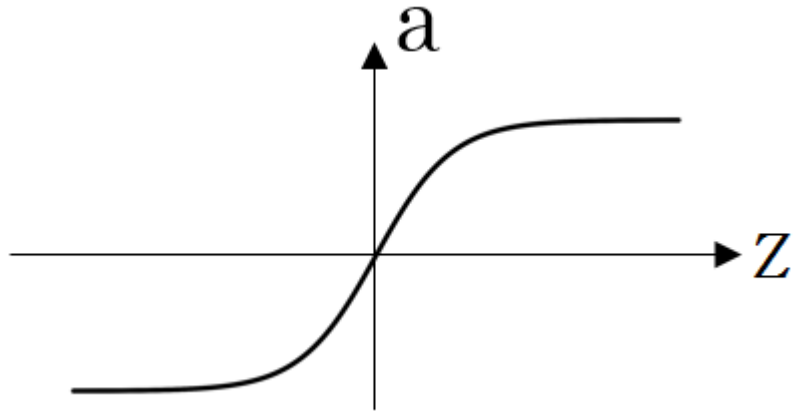


image: Deep learning, Andrew Ng

$$a = \tanh(z) = \frac{\exp(z) - \exp(-z)}{\exp(z) + \exp(-z)}$$

- The output is zero-centered(-1~+1).
- It is simply a scaled sigmoid function.

$$\tanh(z) = 2\sigma(2z) - 1$$

ReLU (Rectified Linear Unit)

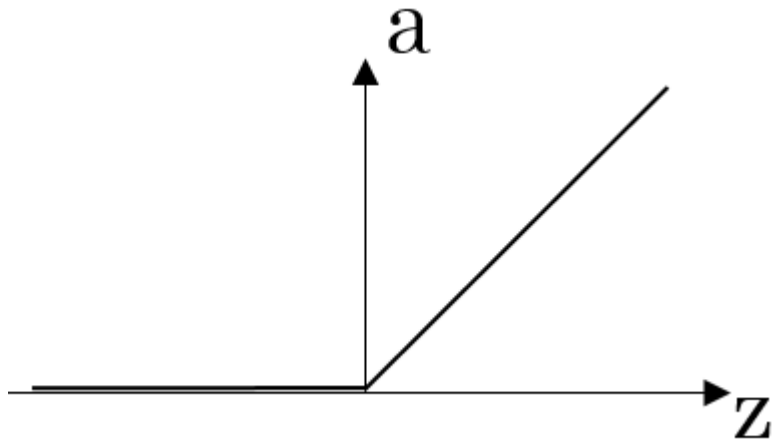


image: Deep learning, Andrew Ng

$$a = \text{ReLU}(z) = \max(0, z)$$

- It is zero when z is ≤ 0
- It is linear with slope 1 when z is > 0 .
- Fast to compute
- Gradients do not vanish for $z > 0$.
- Faster convergence in practice.
- Not zero-centered.
- It is most used in CNN.
- Some neurons can die if it is not activated, never updated.

Leaky ReLU

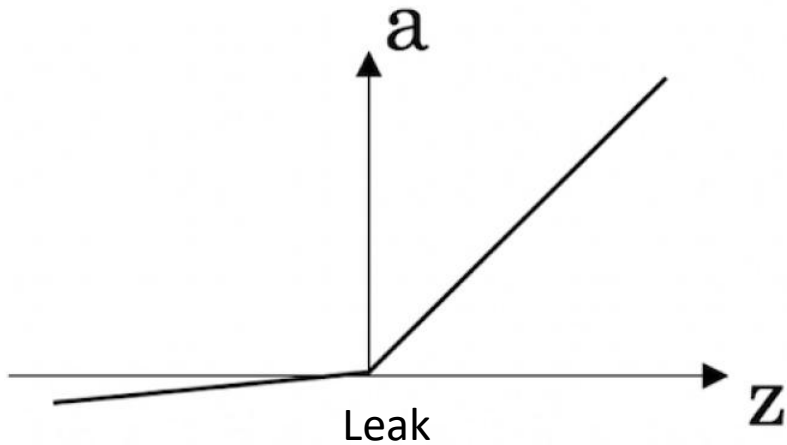


image: Deep learning, Andrew Ng

$$a = \text{Leaky ReLU}(z) = \max(0.01z, z)$$

- It is linear with small negative slope when z is ≤ 0
- It is linear with slope 1 when z is > 0 .
- neurons will not die

ELU(Exponential Linear Unit)

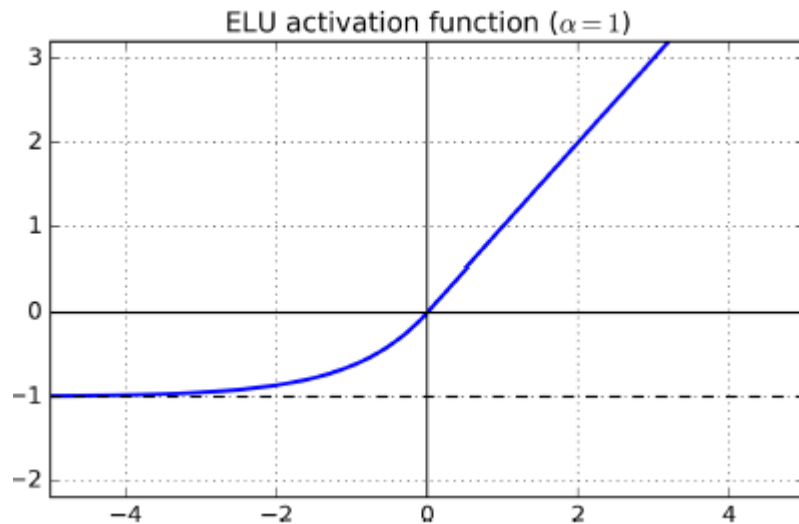


image: Hands-On ML, Aurelien Geron

$$a = ELU(z) = \begin{cases} \alpha(\exp(z) - 1) & \text{if } z < 0 \\ z & \text{if } z \geq 0 \end{cases}$$

- Nonzero gradient for $z < 0 \rightarrow$ avoid dying neurons issue
- It is smooth everywhere including $z=0$

Other Activation functions

- GELU: Gaussian error linear unit
- SwiGLU: Swish-gated linear unit (or sigmoid-gated linear unit)

Softmax function and softmax classifier

$$s_k(x) = W^k \cdot x$$

for each class k

$$p_k = \frac{\exp(s_k(x))}{\sum_{j=1}^K \exp(s_j(x))}$$

- Multiclass classifier
- Provides normalized class probabilities

$$\hat{y} = \operatorname{argmax}_k p_k$$

It predicts the class with the highest estimated probability.

Xavier initialization

When using sigmoid activation function

Multiplies weights by $\frac{\sqrt{2}}{\sqrt{n_{inputs} + n_{outputs}}}$

Source: Hands-On ML, Aurelien Geron

Initialization for Hyperbolic tangent

When using tanh activation function

Multiplies weights by $4 \frac{\sqrt{2}}{\sqrt{n_{inputs} + n_{outputs}}}$

Source: Hands-On ML, Aurelien Geron

He initialization

When using ReLU activation function (and its variants, including the ELU activation)

Multiplies weights by $\sqrt{2} \frac{\sqrt{2}}{\sqrt{n_{inputs} + n_{outputs}}}$

Source: Hands-On ML, Aurelien Geron

Summary: Weights initialization

Activation function	Uniform distribution $[-r, r]$	Normal distribution
Logistic	$r = \sqrt{\frac{6}{n_{\text{inputs}} + n_{\text{outputs}}}}$	$\sigma = \sqrt{\frac{2}{n_{\text{inputs}} + n_{\text{outputs}}}}$
Hyperbolic tangent	$r = \sqrt{\frac{6}{n_{\text{inputs}} + n_{\text{outputs}}}}$	$\sigma = \sqrt{\frac{2}{n_{\text{inputs}} + n_{\text{outputs}}}}$
ReLU (and its variants)	$r = \sqrt{2} \sqrt{\frac{6}{n_{\text{inputs}} + n_{\text{outputs}}}}$	$\sigma = \sqrt{2} \sqrt{\frac{2}{n_{\text{inputs}} + n_{\text{outputs}}}}$

Source: Hands-On ML, Aurelien Geron

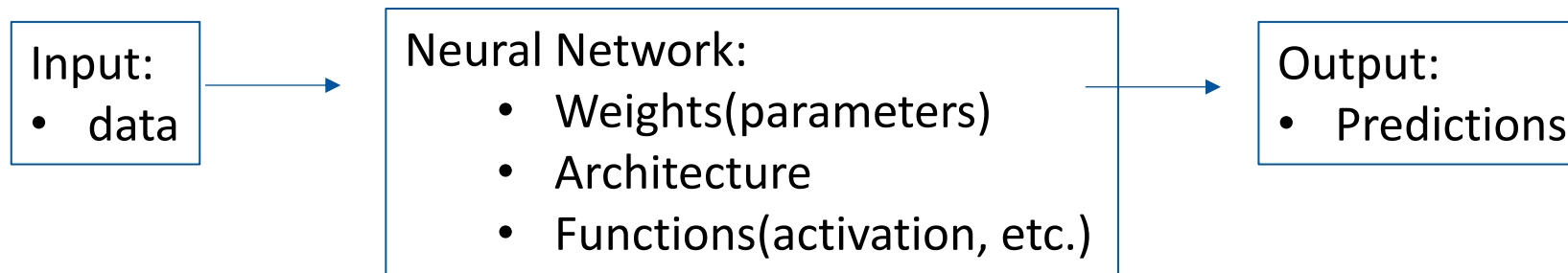
Today's Topics(3)

- Forward propagation

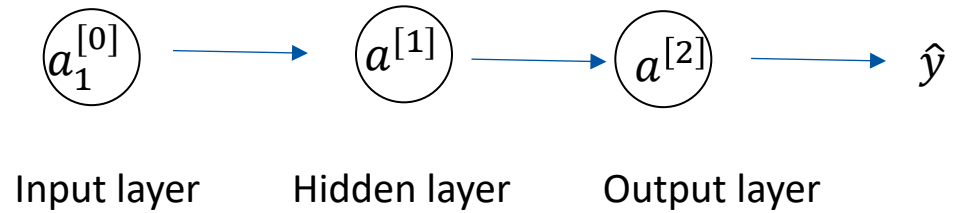
Forward propagation

- The inputs are fed into the neural network.
- This results in a forward cascade of computations across the layers, using the current set of weights.
- The final predicted output can be compared to the true output.

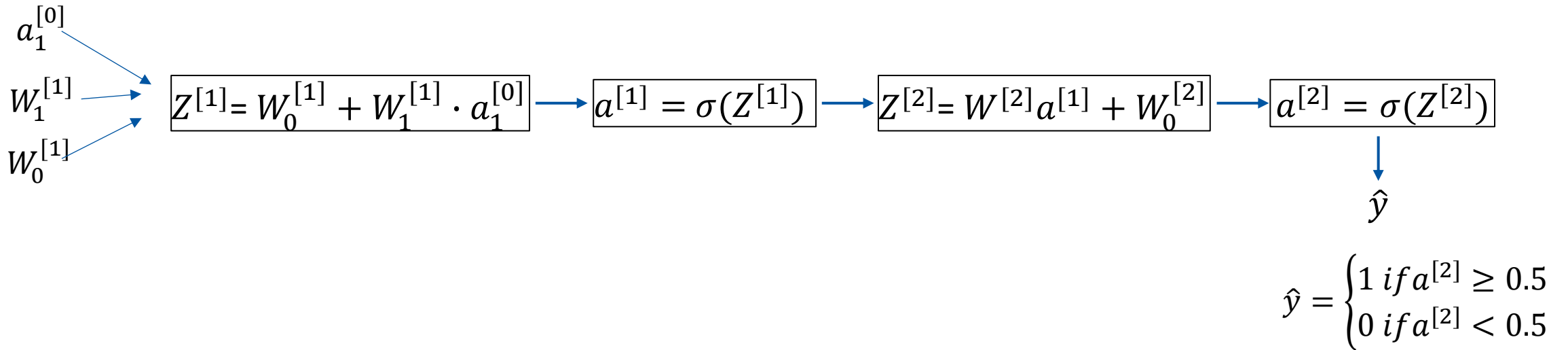
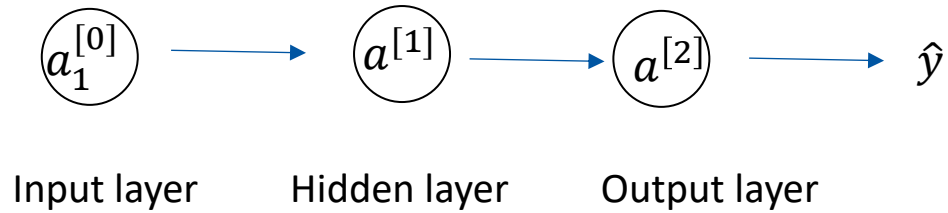
Forward propagation



Forward pass for case1



Forward pass for case1



Forward pass for case2

1st layer

$$Z_1^{[1]} = W_{10}^{[1]} + W_{11}^{[1]} \cdot a_1^{[0]} + W_{12}^{[1]} \cdot a_2^{[0]}$$

$$Z_2^{[1]} = W_{20}^{[1]} + W_{21}^{[1]} \cdot a_1^{[0]} + W_{22}^{[1]} \cdot a_2^{[0]}$$

$$a_1^{[1]} = \sigma(Z_1^{[1]})$$

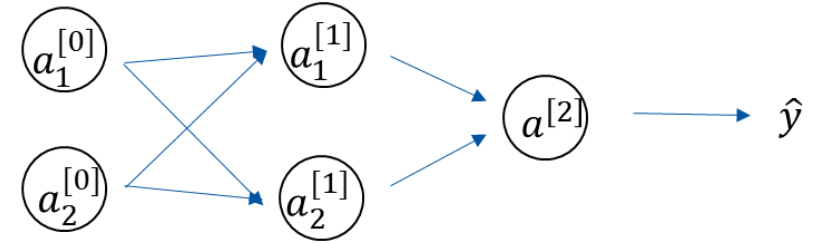
$$a_2^{[1]} = \sigma(Z_2^{[1]})$$

2nd layer

$$Z^{[2]} = W_0^{[2]} + W_1^{[2]} \cdot a_1^{[1]} + W_2^{[2]} \cdot a_2^{[1]}$$

$$a^{[2]} = \sigma(Z^{[2]})$$

$$\hat{y} = \begin{cases} 1 & \text{if } a^{[2]} \geq 0.5 \\ 0 & \text{if } a^{[2]} < 0.5 \end{cases}$$

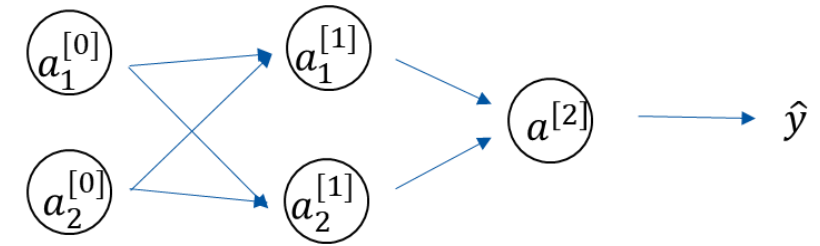
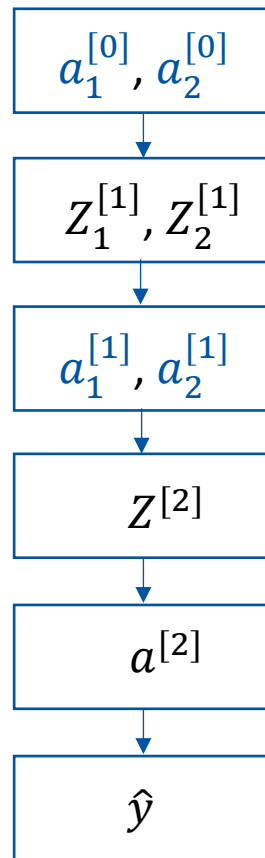


Input layer

Hidden layer

Output layer

Forward pass for case2

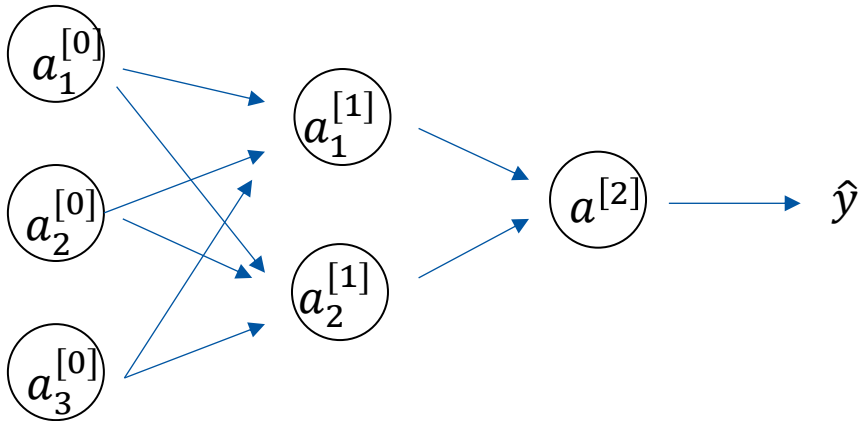


Input layer

Hidden layer

Output layer

Forward pass for case3



Input layer Hidden layer Output layer

$$\hat{y} = \begin{cases} 1 & \text{if } a^{[2]} \geq 0.5 \\ 0 & \text{if } a^{[2]} < 0.5 \end{cases}$$

- Input layer : 3 neurons
- Hidden layer : 2 neurons
- output layer : 1 neurons

Accuracy in Classification

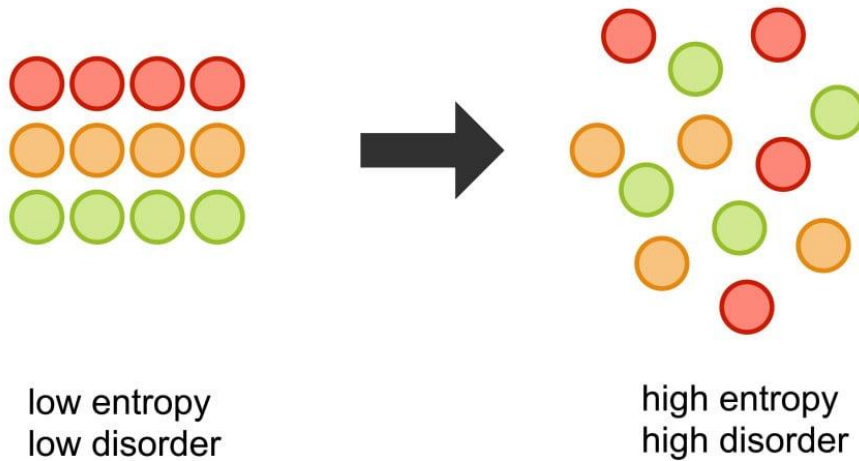
Accuracy

- *Sum(Prediction == Actual Y) / Total number of data*
- *mean(Prediction == Actual Y)*100*

$$prediction = \begin{cases} 1 & \text{if } a^{[2]} \geq 0.5 \\ 0 & \text{if } a^{[2]} < 0.5 \end{cases}$$

Entropy in thermodynamics

- Entropy is a concept of thermodynamics, introduced by Rudolf Clausius, a German physicist, in 1850.
- Entropy is a measurement of the disorder or randomness of a system.



Entropy in information

Claude Shannon introduced the concept of information entropy in his 1948 paper, “A Mathematical Theory of Communication”.

According to Shannon, **entropy** is the average number of bits required to represent or transmit an event drawn from the probability distribution for the random variable.

In other word, the **entropy** of a random variable is the average level of “information”.

In simple terms, entropy indicates the amount of uncertainty of an event.

Source: <https://neptune.ai/blog/cross-entropy-loss-and-its-applications-in-deep-learning>
<https://machinelearningmastery.com/cross-entropy-for-machine-learning/>
<https://www.v7labs.com/blog/cross-entropy-loss-guide>

Entropy in information

Information $h(x)$ can be calculated for an event x , given the probability of the event $P(x)$ as follows:

$$h(x) = -\log_2(P(x))$$

where $\log_2$ is the base-2 logarithm, meaning that the results are in bits.

For the entropy equation for the all events:

$$H(x) = -\sum_i P(x = i) \cdot \log_2(P(x = i))$$

Cross entropy

In the data science domain, the cross entropy between two discrete probability distributions is a metric that captures how similar the two distributions are.

Given the probabilities of the events from P and Q ,

Cross-entropy is following :

$$H(P, Q) = - \sum_x P(x) \cdot \log(Q(x))$$

where $P(x)$ is the probability of the event x in P , $Q(x)$ is the probability of event x in Q

$$H(y, \hat{y}) = - \sum_{j=0}^{N-1} y_j \log(\hat{y}_j)$$

Cross entropy in binary case

$$H(y, \hat{y}) = - \sum_{i \in \{0,1\}} y(i) \cdot \log(\hat{y}(i))$$

$$H(y, \hat{y}) = -[y_0 \cdot \log \hat{y}_0 + y_1 \cdot \log \hat{y}_1]$$

$$H(y, \hat{y}) = -[y \cdot \log \hat{y} + (1 - y) \cdot \log(1 - \hat{y})]$$

when $i = 1$; $y(i) = y, \hat{y}(i) = \hat{y}$

when $i = 0$; $y(i) = 1 - y, \hat{y}(i) = 1 - \hat{y}$

$$\hat{y} = \begin{cases} 1 & \text{if } a^{[2]} \geq 0.5 \\ 0 & \text{if } a^{[2]} < 0.5 \end{cases}$$

Cost function(binary cross entropy)

Cost function J

$$J = -\frac{1}{m} \sum_{i=1}^m [y^i \log(\hat{y}^i) + (1 - y^i) \log(1 - \hat{y}^i)]$$

$$\text{If } y = 1 \quad J = -\frac{1}{m} \sum_{i=1}^m [\log(\hat{y}^i)]$$

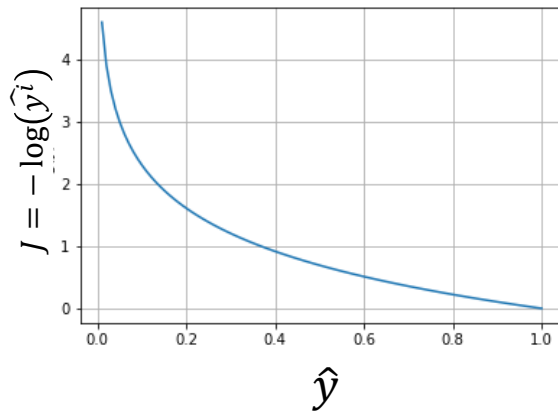
$$\text{If } y = 0 \quad J = -\frac{1}{m} \sum_{i=1}^m [\log(1 - \hat{y}^i)]$$

Cost function(binary cross entropy)

Given x , want $\hat{y}^i \approx y^i$

$$J = -[y^i \log(\hat{y}^i) + (1 - y^i) \log(1 - \hat{y}^i)]$$

If $y = 1$, $J = -\log(\hat{y}^i)$



If $y = 0$, $J = -\log(1 - \hat{y}^i)$

